

Лабораторная работа №4

Создание векторного квантизатора

Векторная квантизация — это метод квантизации, в котором входные данные представляются фиксированным количеством представительных точек. Этот подход эквивалентен округлению чисел. Он широко применяется в ряде областей, таких как распознавание изображений, семантический анализ и наука о данных. Рассмотрим конкретный пример использования искусственных нейронных сетей для построения векторного квантизатора.

Создайте новый файл Python и импортируйте следующие файлы.

```
import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl
```

Загрузим входные данные из файла `data_vector_quantization.txt`. Каждая строка этого файла содержит шесть чисел. Первые два числа образуют точку данных, тогда как последние четыре — прямое кодирование метки. Всего имеются четыре класса данных.

```
# Загрузка входных данных
text = np.loadtxt('data_vector_quantization.txt')
```

Разделим текст на данные и метки.

```
# Разделение текста на данные и метки
data = text[:, 0:2]
labels = text[:, 2:]
```

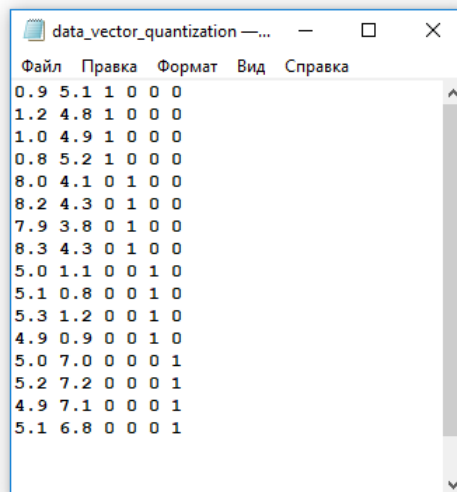


Рис. 14. Файл `data_vector_quantization.txt`

Attention!

В связи с обнаруженной ошибкой, необходимо отредактировать `neurolab/net.py` и изменить строку 127 с

```
layer_out[np['w'][n][st:i]].fill(1.0)
```

на

```
layer_out[np['w'][n][int(st):int(i)]].fill(1.0)
```

Определим нейронную сеть с двумя слоями: десять нейронов во входном слое и четыре в выходном.

```
# Определение нейронной сети с двумя слоями:
# десять нейронов во входном слое и четыре в выходном
num_input_neurons = 10
num_output_neurons = 4
weights = [1/num_output_neurons] * num_output_neurons
nn = nl.net.newlvq(nl.tool.minmax(data), num_input_neurons, weights)
```

Обучим нейронную сеть, используя тренировочный набор данных.

```
# Обучение нейронной сети
_ = nn.train(data, labels, epochs=500, goal=-1)
```

Создадим сетку точек для визуализации кластеров.

```
# Создание входной сетки данных
xx, yy = np.meshgrid(np.arange(0, 10, 0.2),
                     np.arange(0, 10, 0.2))
xx.shape = xx.size, 1
yy.shape = yy.size, 1
grid_xy = np.concatenate((xx, yy), axis=1)
```

Выполним вычисления над сеткой точек данных с помощью нейронной сети.

```
# Выполнение вычислений над входной сеткой данных
grid_eval = nn.sim(grid_xy)
```

Извлечем четыре класса.

```
# Определение четырех классов
class_1 = data[labels[:,0] == 1]
class_2 = data[labels[:,1] == 1]
class_3 = data[labels[:,2] == 1]
class_4 = data[labels[:,3] == 1]
```

Извлечем сетки, соответствующие этим четырем классам.

```
# Define X-Y grids for all the 4 classes
grid_1 = grid_xy[grid_eval[:,0] == 1]
grid_2 = grid_xy[grid_eval[:,1] == 1]
grid_3 = grid_xy[grid_eval[:,2] == 1]
grid_4 = grid_xy[grid_eval[:,3] == 1]
```

Построим график результирующих данных.

```
# Построение графика результирующих данных
plt.plot(class_1[:,0], class_1[:,1], 'ko',
         class_2[:,0], class_2[:,1], 'ko',
```

```

class_3[:,0], class_3[:,1], 'ko',
class_4[:,0], class_4[:,1], 'ko')
plt.plot(grid_1[:,0], grid_1[:,1], 'm.',
grid_2[:,0], grid_2[:,1], 'bx',
grid_3[:,0], grid_3[:,1], 'c^',
grid_4[:,0], grid_4[:,1], 'y+')
plt.axis([0, 10, 0, 10])
plt.xlabel('Размерность 1')
plt.ylabel('Размерность 2')
plt.title('Векторная квантизация')

plt.show()

```

Полный код примера содержится в файле `vector_quantizer.py`. В процессе выполнения этого кода на экране отобразится следующий график, представляющий входные точки данных и границы между кластерами (рис. 14.10).

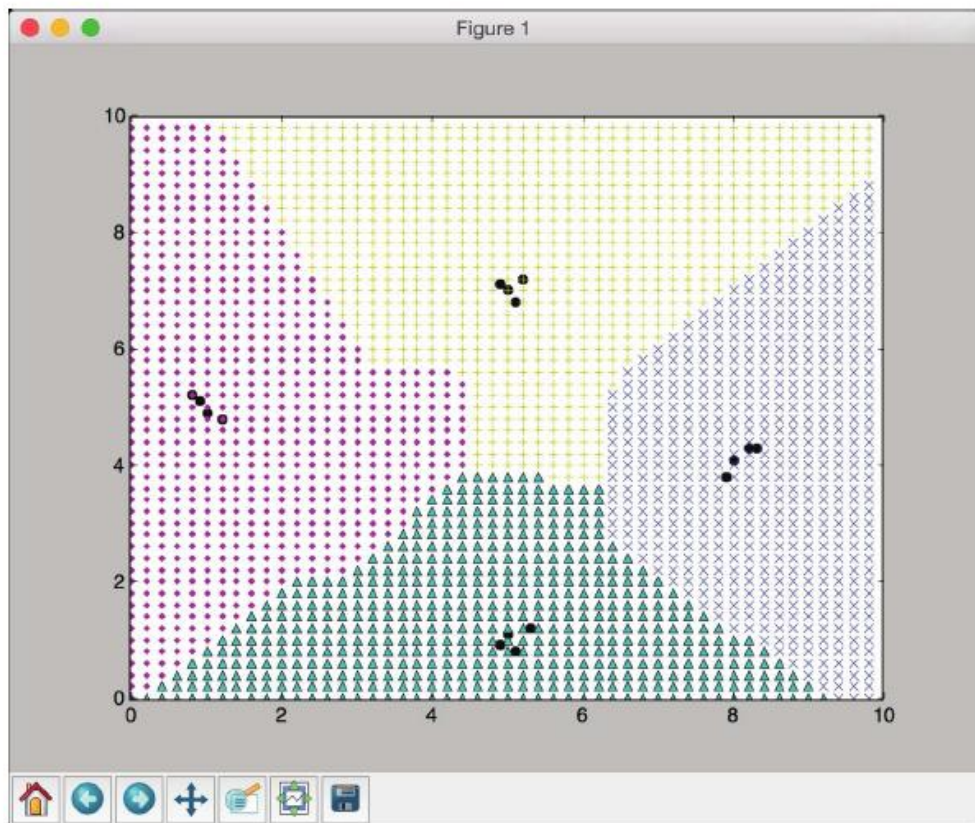


Рис. 14.10. входные точки данных и границы между кластерами

В окне терминала отобразится следующая информация (рис. 14.11).

```
Epoch: 100; Error: 0.0;  
Epoch: 200; Error: 0.0;  
Epoch: 300; Error: 0.0;  
Epoch: 400; Error: 0.0;  
Epoch: 500; Error: 0.0;  
The maximum number of train epochs is reached
```

Рис. 14.11

Содержание работы.

Ответьте на вопросы и выполните следующие задания:

- 1. Что называют векторной квантизацией?**
- 2. В каких областях применяется векторная квантизация?**
- 3. Привести код на Python векторного квантизатора на основе нейронной сети, рассмотренной в методических указаниях.**
- 4. Самостоятельно провести эксперименты с векторным квантизатором на иных произвольных входных данных (файл `data_vector_quantization.txt`). Представить графически входные точки данных и границы между кластерами.**

```

import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl

# Load input data
text = np.loadtxt('data_vector_quantization.txt')

# Separate it into data and labels
data = text[:, 0:2]
labels = text[:, 2:]

# Define a neural network with 2 layers:
# 10 neurons in input layer and 4 neurons in output layer
num_input_neurons = 10
num_output_neurons = 4
weights = [1/num_output_neurons] * num_output_neurons
nn = nl.net.newlvq(nl.tool.minmax(data), num_input_neurons, weights)

# Train the neural network
_ = nn.train(data, labels, epochs=500, goal=-1)

# Create the input grid
xx, yy = np.meshgrid(np.arange(0, 10, 0.2), np.arange(0, 10, 0.2))
xx.shape = xx.size, 1
yy.shape = yy.size, 1
grid_xy = np.concatenate((xx, yy), axis=1)

# Evaluate the input grid of points
grid_eval = nn.sim(grid_xy)

# Define the 4 classes
class_1 = data[labels[:,0] == 1]
class_2 = data[labels[:,1] == 1]
class_3 = data[labels[:,2] == 1]
class_4 = data[labels[:,3] == 1]

# Define X-Y grids for all the 4 classes
grid_1 = grid_xy[grid_eval[:,0] == 1]
grid_2 = grid_xy[grid_eval[:,1] == 1]
grid_3 = grid_xy[grid_eval[:,2] == 1]
grid_4 = grid_xy[grid_eval[:,3] == 1]

# Plot the outputs
plt.plot(class_1[:,0], class_1[:,1], 'ko',
         class_2[:,0], class_2[:,1], 'ko',
         class_3[:,0], class_3[:,1], 'ko',
         class_4[:,0], class_4[:,1], 'ko')
plt.plot(grid_1[:,0], grid_1[:,1], 'm.',
         grid_2[:,0], grid_2[:,1], 'bx',
         grid_3[:,0], grid_3[:,1], 'c^',
         grid_4[:,0], grid_4[:,1], 'y+')
plt.axis([0, 10, 0, 10])
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.title('Vector quantization')

plt.show()

```

Рис. 14.П1. Пример кода векторного квантизатора на Python

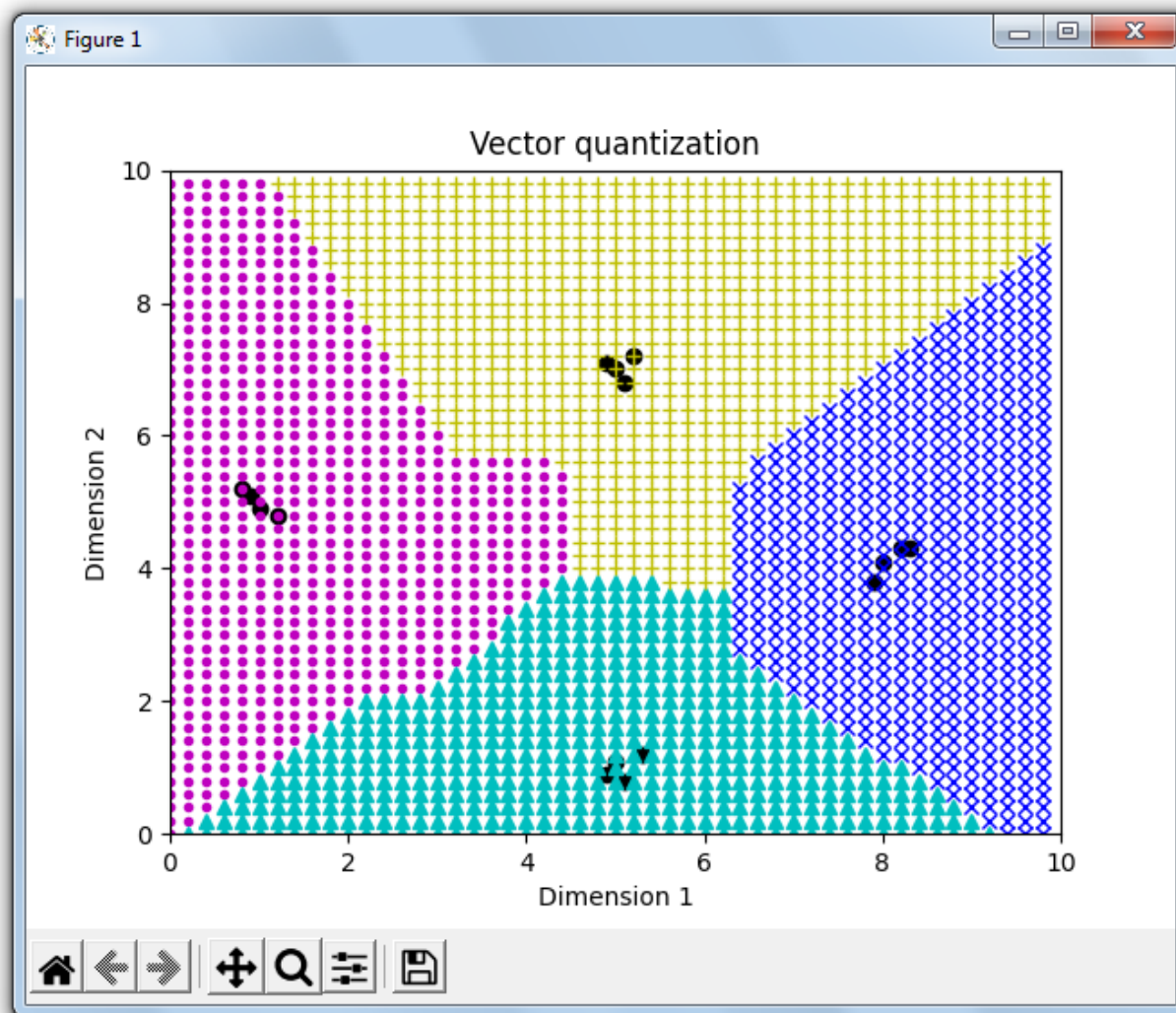


Рис. 14.П2. Входные точки данных и границы между кластерами

```
Epoch: 100; Error: 0.0;  
Epoch: 200; Error: 0.0;  
Epoch: 300; Error: 0.0;  
Epoch: 400; Error: 0.0;  
Epoch: 500; Error: 0.0;  
The maximum number of train epochs is reached
```

Рис. 14.П3. Результат выполнения кода